

Unit-II: Server-Side Programming with Python (8 hours)

Compiled By

Lecturer, Sunil Bahadur Bist

NAST, Dhangadhi

Working with Python Data Structures

- Lists: `mylist = ["apple", "banana", "cherry"]`
- Dictionaries: `thisdict = {"brand": "Ford", "model": "Mustang", "year": 1964}`
- Tuples: `mytuple = ("apple", "banana", "cherry")` and
- Sets: `myset = {"apple", "banana", "cherry"}`

Lists

- Lists are used to store multiple items in a single variable.
- Lists are created using square brackets: []
- List items are ordered, changeable, and allow duplicate values.
- When we say that lists are ordered, it means that the items have a defined order, and that order will not change.
- If you add new items to a list, the new items will be placed at the end of the list.
- List items are indexed, the first item has index [0], the second item has index [1] etc.

Lists

- To determine how many items a list has, use the len() function

```
print(len(mylist))
```

- List, tuple, set items can be of any data type:

```
list1 = ["apple", "banana", "cherry"]
```

```
list2 = [1, 5, 7, 9, 3]
```

```
list3 = [True, False, False]
```

- A list, tuple, set can contain different data types:

```
list1 = ["abc", 34, True, 40, "male"]
```

Lists

- It is also possible to use the list() constructor when creating a new list.

```
mylist = list(("apple", "banana", "cherry"))
```

```
# note the double round-brackets
```

```
print(mylist)
```

- List items are indexed and you can access them by referring to the index number:

```
thislist = ["apple", "banana", "cherry"]
```

```
print(thislist[1])
```

Lists

- Negative indexing means start from the end
- -1 refers to the last item, -2 refers to the second last item etc.

```
thislist = ["apple", "banana", "cherry"]  
print(thislist[-1])
```

- List, tuple, set supports negative indexing

Lists

- To insert a new list item, without replacing any of the existing values, we can use the insert() method.
- The insert() method inserts an item at the specified index:

```
thislist = ["apple", "banana", "cherry"]  
thislist.insert(2, "watermelon")  
print(thislist)
```

Lists

- To add an item to the end of the list, use the `append()` method:

```
thislist = ["apple", "banana", "cherry"]
```

```
thislist.append("orange")
```

```
print(thislist)
```


Lists

- The remove() method removes the specified item.

```
thislist = ["apple", "banana", "cherry"]
```

```
thislist.remove("banana")
```

```
print(thislist)
```

Lists

- The pop() method removes the specified index.

```
thislist = ["apple", "banana", "cherry"]  
thislist.pop(1)  
print(thislist)
```

- If you do not specify the index, the pop() method removes the last item.

```
thislist = ["apple", "banana", "cherry"]  
thislist.pop()  
print(thislist)
```

Dictionaries

- Dictionaries are used to store data values in key:value pairs.
- A dictionary is a collection which is ordered*, changeable and do not allow duplicates.
- As of Python version 3.7, dictionaries are ordered. In Python 3.6 and earlier, dictionaries are unordered.
- Dictionaries are written with curly brackets, and have keys and values:
{ }
- It is also possible to use the dict() constructor to make a dictionary.

Dictionaries

```
thisdict = dict(name = "John", age = 36, country = "Norway")  
print(thisdict)
```

- You can access the items of a dictionary by referring to its key name, inside square brackets:

```
x = thisdict["model"]
```

- There is also a method called `get()` that will give you the same result:

```
x = thisdict.get("model")
```

Dictionaries

- The `keys()` method will return a list of all the keys in the dictionary.

```
x = thisdict.keys()
```

- You can change the value of a specific item by referring to its key name:

```
thisdict["year"] = 2018
```

- The `update()` method will update the dictionary with the items from the given argument.
- The argument must be a dictionary, or an iterable object with key:value pairs.

```
thisdict.update({"year": 2020})
```

Dictionaries

- Adding an item to the dictionary is done by using a new index key and assigning a value to it:

```
    thisdict["color"] = "red"  
    print(thisdict)
```

- You can also use the values() method to return values of a dictionary:

```
    for x in thisdict.values():  
        print(x)
```

- You can use the keys() method to return the keys of a dictionary:

Dictionaries

```
for x in thisdict.keys():  
    print(x)
```

- Loop through both keys and values, by using the items() method:

```
for x, y in thisdict.items():  
    print(x, y)
```

Tuples

- Tuples are used to store multiple items in a single variable.
- A tuple is a collection which is ordered and unchangeable.
- Tuples are written with round brackets. ()

```
mytuples = ("apple", "banana", "cherry")  
print(mytuples)
```

- Tuple items are ordered, unchangeable, and allow duplicate values.
- Tuple items are indexed, the first item has index [0], the second item has index [1] etc.

Tuples

- One item tuple, remember the comma:

```
thistuple = ("apple",)  
print(type(thistuple))
```

- #NOT a tuple

```
thistuple = ("apple")  
print(type(thistuple))
```

Tuples

- It is also possible to use the tuple() constructor to make a tuple.

```
thistuple = tuple(("apple", "banana", "cherry"))
```

```
# note the double round-brackets
```

```
print(thistuple)
```

- You can access tuple items by referring to the index number, inside square brackets:

```
print(thistuple[1])
```

Tuples

- Convert the tuple into a list to be able to change it:

```
x = ("apple", "banana", "cherry")
```

```
y = list(x)
```

```
y[1] = "kiwi"
```

```
x = tuple(y)
```

```
print(x)
```

Tuples

- Convert the tuple into a list, remove "apple", and convert it back into a tuple:

```
thistuple = ("apple", "banana", "cherry")
```

```
y = list(thistuple)
```

```
y.remove("apple")
```

```
thistuple = tuple(y)
```

Sets

- Sets are used to store multiple items in a single variable.
- A set is a collection which is unordered, unchangeable*, and unindexed.
- * Note: Set items are unchangeable, but you can remove items and add new items.
- Sets are written with curly brackets. {}
- Set items are unordered, unchangeable, and do not allow duplicate values.
- It is also possible to use the set() constructor to make a set.

```
my_set = set((1, 2, 2, 3, 4, 4, 5))  
print(my_set)
```

Sets

- To remove an item in a set, use the `remove()`, or the `discard()` method.

```
thisset = {"apple", "banana", "cherry"}  
thisset.remove("banana")  
print(thisset)
```
- Once a set is created, you cannot change its items, but you can add new items.

```
thisset = {"apple", "banana", "cherry"}  
thisset.add("orange")  
print(thisset)
```

Sets

- To add items from another set into the current set, use the `update()` method.

```
thisset = {"apple", "banana", "cherry"}  
tropical = {"pineapple", "mango", "papaya"}  
thisset.update(tropical)  
print(thisset)
```

Working with Python Data Structures

Assignment

- Differences between lists, tuples, and sets